

Tarea 1

Eckert Gibreel Baeza Gómez

19 de Agosto 2026

1 Información

1.1 ¿Qué es la información?

En el contexto de la informática, podemos entenderla como datos organizados y significativos que pueden ser interpretados, procesados y almacenados por sistemas. De manera simple, podemos definirla como el resultado de procesar datos para revelar su significado.

1.2 Características de la información

- **Exactitud:** La información debe representar correctamente la realidad y estar basada en datos sin errores.
- **Relevancia:** La información debe ser pertinente para el problema o propósito. Esto quiere decir que no toda la información disponible es útil para una determinada tarea.
- **Actualidad:** La información debe estar disponible cuando se necesita y reflejar, de acuerdo con el contexto, una situación suficientemente actual.
- **Compleitud:** Debe contener los datos necesarios para el propósito. Si faltan datos importantes, podríamos llegar a una conclusión incorrecta.
- **Consistencia:** Los datos no deben contradecirse entre sí y deben mantener coherencia entre las diferentes fuentes o partes del sistema.
- **Confiabilidad:** La información debe provenir de datos y procesos en los que podamos confiar. Esto está relacionado con la calidad de la fuente, la forma en que se obtuvieron los datos y el procesamiento utilizado.
- **Accesibilidad:** La información debe poder ser obtenida y utilizada por quienes tienen autorización para hacerlo.
- **Interpretabilidad:** La información debe presentarse de una manera que permita al usuario comprender su significado.

2 Modelos de Datos

2.1 Orientado a Objetos

Es un modelo en el que la información se representa mediante objetos que combinan datos, comportamientos e identidad propia. Los objetos pueden relacionarse entre sí y mantenerse de forma persistente, permitiendo que la base de datos y la aplicación trabajen bajo un mismo paradigma orientado a objetos.

2.1.1 Características

- **Objetos complejos:** Permite representar estructuras formadas por diferentes tipos de datos, como listas, arreglos, conjuntos y objetos anidados.
- **Identidad de objetos:** Cada objeto tiene una identidad propia, independiente de los valores de sus atributos.
- **Encapsulación:** Los datos y las operaciones relacionadas con ellos pueden mantenerse dentro del mismo objeto.
- **Herencia:** Permite que una clase herede atributos y comportamientos de otra.
- **Polimorfismo:** Permite que una misma operación tenga diferentes comportamientos dependiendo del objeto.
- **Persistencia:** Los objetos pueden conservarse en la base de datos después de finalizar la ejecución de la aplicación.
- **Relaciones entre objetos:** Los objetos pueden mantener relaciones directas entre sí.

2.1.2 Ventajas

- **Representación natural:** Permite representar directamente objetos complejos de una aplicación.
- **Integración con lenguajes orientados a objetos:** Facilita el trabajo con lenguajes como Java y C++.
- **Menor impedancia objeto-relacional:** Reduce la necesidad de convertir objetos en tablas y viceversa.
- **Encapsulación:** Permite mantener los datos y las operaciones relacionadas dentro del mismo objeto.
- **Herencia y reutilización:** Permite reutilizar estructuras y comportamientos mediante clases.
- **Manejo de datos complejos:** Es adecuado para información como multimedia, simulaciones y estructuras anidadas.

2.1.3 Desventajas

- **Menor adopción:** El modelo relacional continúa siendo más utilizado en aplicaciones empresariales.
- **Falta de un estándar único:** Existen diferentes implementaciones y extensiones dependiendo del sistema.
- **Curva de aprendizaje:** Es necesario comprender conceptos de bases de datos y programación orientada a objetos.
- **Interoperabilidad limitada:** Las diferencias entre implementaciones pueden dificultar la migración entre sistemas.
- **Sobrecarga:** Algunas operaciones pueden resultar más costosas que en modelos diseñados específicamente para datos tabulares.

2.1.4 ¿Cuándo utilizarlo?

Se utiliza cuando una aplicación trabaja principalmente con objetos complejos y relaciones entre ellos, especialmente cuando estos objetos necesitan mantenerse de forma persistente. Es adecuado para aplicaciones como simulaciones científicas, sistemas multimedia, sistemas geográficos y aplicaciones que manejan estructuras de datos complejas.

2.1.5 Software que lo implementa

- **ObjectDB:** Base de datos orientada a objetos para Java que permite almacenar y consultar objetos Java directamente.
- **ObjectStore:** Permite almacenar objetos persistentes complejos y mantener sus relaciones.
- **GemStone/S:** Plataforma orientada a objetos basada en Smalltalk que permite almacenar objetos de forma persistente.
- **Versant:** Base de datos orientada a objetos utilizada para gestionar estructuras complejas de objetos relacionados.
- **db4o:** Base de datos orientada a objetos para Java y .NET que permite almacenar objetos directamente.

2.2 Columnar

Es un sistema donde la información se organiza y guarda por columnas en lugar de por filas. Así, cuando una consulta necesita ciertos datos, puede leer únicamente las columnas necesarias sin procesar toda la información. Además, como los datos de una misma columna suelen ser similares, pueden comprimirse fácilmente, haciendo más eficiente el almacenamiento y la lectura de grandes cantidades de datos.

2.2.1 Características

- Almacena los datos por columnas en lugar de por filas (como en el modelo relacional tradicional).
- Cada columna se guarda como un bloque independiente, lo que facilita la compresión y la lectura selectiva.
- Está optimizado para consultas analíticas que involucran pocas columnas pero muchas filas (agregaciones, sumas, promedios).
- Los datos se ordenan y agrupan por columna, lo que mejora el rendimiento en escenarios de lectura masiva.

2.2.2 Ventajas

- Alto rendimiento en consultas OLAP (procesamiento analítico en línea) y *big data*.
- Eficiencia en compresión: al tener datos homogéneos por columna, se comprimen mucho mejor que en almacenamiento por filas.
- Menor consumo de E/S: solo se leen las columnas necesarias para la consulta.
- Escalabilidad horizontal: muchos sistemas columnar están diseñados para distribuirse en clústeres.

2.2.3 Desventajas

- Bajo rendimiento en operaciones de escritura frecuentes o transacciones OLTP (inserciones, actualizaciones puntuales).
- No es adecuado para aplicaciones que requieren recuperar filas completas con muchos campos.
- Mayor complejidad en el mantenimiento de índices y actualizaciones.
- Curva de aprendizaje si se viene de bases de datos relacionales tradicionales.

2.2.4 Casos de uso

El modelo columnar se utiliza principalmente cuando se necesita analizar grandes volúmenes de información, especialmente en sistemas de inteligencia de negocios. Es adecuado cuando las consultas suelen trabajar con pocas columnas de una gran cantidad de registros, por ejemplo, para realizar sumas, promedios, filtros o identificar tendencias.

También resulta útil en el análisis de registros, series temporales y aplicaciones de ciencia de datos, donde se realizan consultas y agregaciones sobre grandes cantidades de información.

No es la mejor opción para sistemas que realizan muchas operaciones de inserción, actualización o consulta de registros individuales, como ocurre frecuentemente en sistemas transaccionales.

2.2.5 Software que lo implementa

- **Apache Cassandra:** Sistema distribuido de tipo *wide-column* diseñado para manejar grandes volúmenes de datos con alta disponibilidad y escalabilidad.
- **Apache HBase:** Base de datos distribuida de tipo *wide-column* que se ejecuta sobre Hadoop y HDFS. Está diseñada para almacenar y consultar grandes cantidades de datos.
- **Google Bigtable:** Base de datos distribuida de tipo *wide-column* desarrollada por Google para almacenar grandes volúmenes de datos y ofrecer alta escalabilidad.
- **Apache Parquet:** Formato de almacenamiento columnar, no una base de datos. Organiza los datos por columnas y permite almacenarlos de forma eficiente, especialmente para análisis de grandes volúmenes.
- **ClickHouse:** Sistema de gestión de bases de datos columnar de código abierto, optimizado para realizar consultas analíticas rápidas sobre grandes cantidades de datos.
- **Amazon Redshift:** Servicio de almacén de datos en la nube de AWS que utiliza almacenamiento columnar para realizar consultas analíticas sobre grandes volúmenes de información.
- **Google BigQuery:** Servicio de análisis de datos completamente administrado de Google Cloud que utiliza almacenamiento columnar para ejecutar consultas sobre grandes conjuntos de datos.

2.3 Documental

Es un modelo en el que la información se organiza y almacena mediante documentos, en lugar de tablas con filas y columnas. Cada documento contiene los datos relacionados de una entidad y puede tener una estructura flexible, normalmente utilizando formatos como JSON o BSON. Esto permite almacenar información que puede variar entre documentos sin requerir un esquema rígido.

2.3.1 Características

- Almacena datos en forma de documentos, generalmente en formato JSON, BSON o XML.
- Cada documento es una estructura jerárquica formada por pares clave-valor, listas y objetos anidados.
- No requiere un esquema fijo: cada documento puede tener campos diferentes, lo que permite un esquema flexible.
- Los documentos se agrupan en colecciones, que cumplen una función similar a las tablas del modelo relacional.
- Orientación a objetos: Los documentos pueden representar de forma natural objetos utilizados por las aplicaciones.
- Estructuras anidadas: Permite almacenar datos relacionados dentro de un mismo documento mediante objetos y listas.
- Identificador único: Cada documento cuenta con un identificador que permite localizarlo de forma directa.

- Consultas e indexación: Permite buscar y filtrar información utilizando los campos contenidos dentro de los documentos.
- Escalabilidad horizontal: Puede distribuir los documentos entre varios servidores para manejar grandes cantidades de información.

2.3.2 Ventajas

- Flexibilidad de esquema: permite evolución rápida del modelo de datos sin migraciones complejas.
- Representación natural de datos semiestructurados y anidados (API REST, JSON).
- Buen rendimiento para operaciones de lectura/escritura de documentos completos.
- Escalabilidad horizontal sencilla mediante *sharding*.
- Índices sobre campos anidados y arrays.

2.3.3 Desventajas

- No es óptimo para relaciones complejas entre documentos (carece de *joins* nativos).
- Puede haber redundancia de datos (denormalización) para evitar múltiples consultas.
- Consumo de almacenamiento mayor debido al overhead del formato JSON/BSON.
- Menor madurez en transacciones ACID comparado con bases relacionales (aunque ha mejorado).

2.3.4 Casos de uso

Se utiliza principalmente en aplicaciones que manejan datos semiestructurados o cuyo esquema puede cambiar con frecuencia. Es adecuado para aplicaciones web y móviles que requieren un desarrollo ágil, así como para catálogos de comercio electrónico, perfiles de usuario y sistemas de gestión de contenido, donde la información puede variar entre registros. También resulta útil en sistemas de Internet de las Cosas, donde se reciben datos heterogéneos de diferentes dispositivos, y en aplicaciones que necesitan acceder rápidamente a información almacenada de forma similar a como será utilizada. En general, es conveniente cuando los datos pueden representarse naturalmente como documentos y no requieren relaciones complejas entre múltiples entidades.

2.3.5 Software que lo implementa

- **MongoDB:** Base de datos documental que almacena información en documentos BSON y permite realizar consultas sobre sus campos.
- **CouchDB:** Base de datos documental que utiliza documentos JSON y facilita la replicación y sincronización de datos.
- **Firebase Firestore:** Base de datos documental en la nube de Google, orientada a aplicaciones web y móviles.
- **Apache Couchbase:** Base de datos documental que combina almacenamiento de documentos con caché y consultas mediante N1QL.
- **RethinkDB:** Base de datos documental diseñada para aplicaciones que requieren actualización de datos en tiempo real.
- **Amazon DocumentDB:** Servicio de base de datos documental de AWS compatible con aplicaciones que utilizan MongoDB.

2.4 Clave-valor

Es un modelo de datos NoSQL que almacena la información en pares formados por una **clave** y un **valor**. La clave identifica de manera única al dato y permite localizarlo directamente, mientras que el valor contiene la información asociada, que puede ser desde un dato simple hasta una estructura más compleja. Su funcionamiento es similar al de un diccionario, donde se utiliza una palabra para obtener rápidamente su definición.

2.4.1 Características

- Modelo más simple: Almacena pares formados por una clave única y un valor asociado.
- Valor flexible: El valor puede ser cualquier tipo de dato, como texto, números, objetos JSON o datos binarios.
- Operaciones básicas: Las principales operaciones son `get(clave)`, `put(clave, valor)` y `delete(clave)`.
- Sin estructura interna: La base de datos no necesita conocer ni interpretar la estructura del valor almacenado.
- Alta velocidad y baja latencia: La búsqueda directa mediante la clave permite realizar operaciones de lectura y escritura rápidamente.
- Escalabilidad horizontal: Los datos pueden distribuirse entre diferentes servidores para manejar mayores volúmenes de información.
- Esquema flexible: No requiere definir una estructura fija antes de almacenar los datos.

2.4.2 Ventajas

- Sencillez extrema: fácil de entender y usar.
- Rendimiento muy alto para lecturas y escrituras por clave.
- Escalabilidad horizontal casi lineal mediante particionamiento (*sharding*) por clave.
- Muy eficiente en memoria y almacenamiento.
- Ideal para almacenar datos efímeros o de acceso frecuente.

2.4.3 Desventajas

- No permite consultas por valor o por atributos internos (a menos que se implementen índices aparte).
- Carece de relaciones, transacciones complejas y *joins*.
- No hay esquema ni validación de tipos.
- Puede llevar a duplicación de datos si se necesitan múltiples vistas.

2.4.4 Casos de uso

El modelo de datos clave-valor se utiliza cuando se necesita acceder a información de forma rápida mediante una clave conocida, especialmente cuando se manejan grandes cantidades de datos y se requiere baja latencia. Es adecuado para aplicaciones donde las operaciones principales consisten en almacenar, consultar, actualizar o eliminar valores asociados a una clave, sin necesidad de realizar relaciones o consultas complejas. Algunos de sus principales casos de uso son:

- Sistemas de caché, donde se requiere recuperar información frecuentemente y con baja latencia.
- Gestión de sesiones de usuario en aplicaciones web, juegos y plataformas en línea.
- Almacenamiento de configuraciones y preferencias que necesitan consultarse o modificarse rápidamente.
- Contadores, colas simples y sistemas de publicación/suscripción que requieren operaciones rápidas.
- Almacenamiento de perfiles de usuario cuando el acceso se realiza principalmente mediante un identificador.

2.4.5 Software que lo implementa

- **Redis:** Base de datos en memoria que ofrece acceso rápido a datos clave-valor y permite utilizar estructuras de datos como listas, conjuntos y hashes.
- **Amazon DynamoDB:** Servicio administrado en la nube que utiliza los modelos clave-valor y documental para almacenar grandes cantidades de datos de forma escalable.
- **Apache Cassandra:** Base de datos distribuida de tipo *wide-column* que también puede utilizarse para almacenar y consultar información mediante claves.
- **etcd:** Almacén distribuido clave-valor utilizado principalmente para guardar configuraciones y coordinar sistemas distribuidos.
- **ZooKeeper:** Sistema distribuido que utiliza pares clave-valor para gestionar configuraciones, coordinación y sincronización entre aplicaciones.
- **Riak:** Base de datos distribuida clave-valor diseñada para ofrecer alta disponibilidad y tolerancia a fallos.
- **Berkeley DB:** Biblioteca embebida que permite integrar almacenamiento clave-valor directamente dentro de una aplicación.

2.5 Orientado a grafos

Un modelo de datos orientado a grafos representa la información mediante nodos y relaciones (aristas), permitiendo almacenar y consultar de forma eficiente datos donde las conexiones entre las entidades son importantes.

2.5.1 Características

- Modela los datos como nodos (entidades) y aristas (relaciones) con propiedades.
- Tanto nodos como aristas pueden tener atributos (pares clave-valor).
- Permite navegar por las relaciones de forma eficiente, sin necesidad de *joins*.
- Soporta consultas de recorrido (ej. *camino más corto*, *amigos en común*).
- Utiliza lenguajes de consulta como **Cypher** (Neo4j) o **Gremlin**.
- Esquema flexible: Permite agregar o modificar nodos, propiedades y relaciones sin requerir una estructura rígida.
- Relaciones explícitas: Las conexiones entre entidades forman parte de los datos y pueden tener sus propias propiedades.
- Análisis de redes: Permite aplicar algoritmos para encontrar patrones, rutas y conexiones entre los datos.

2.5.2 Ventajas

- Representación natural de datos fuertemente interconectados (redes sociales, mapas).
- Consultas de relaciones muy rápidas, incluso con varios niveles de profundidad.
- Flexibilidad de esquema: se pueden añadir nuevos tipos de nodos y relaciones sin migrar.
- Ideales para detectar patrones y hacer análisis de grafos (centralidad, comunidades).
- Evita la explosión de *joins* que ocurriría en un modelo relacional.

2.5.3 Desventajas

- Menor rendimiento para consultas que no aprovechan las relaciones (ej. agregaciones simples).
- Curva de aprendizaje pronunciada (modelo de grafos y lenguajes de consulta específicos).
- Escalabilidad horizontal puede ser compleja, especialmente para grafos muy grandes.
- No todos los motores de grafos son transaccionales con ACID completo.

2.5.4 Casos de uso

El modelo de datos orientado a grafos se utiliza cuando las relaciones entre los datos son especialmente importantes y se necesita analizar cómo se conectan las distintas entidades. Es adecuado para sistemas donde se realizan recorridos sobre múltiples relaciones o se buscan patrones dentro de redes complejas, como ocurre en los siguientes casos:

- Redes sociales y recomendaciones de conexiones.
- Sistemas de detección de fraude (análisis de redes de transacciones).
- Motores de recomendación (productos, contenidos).
- Gestión de redes y rutas (logística, telecomunicaciones).
- Gestión de conocimiento y ontologías (grafos de entidades).

2.5.5 Software que lo implementa

- **Neo4j**: Base de datos orientada a grafos especializada en almacenar y consultar relaciones entre datos. Utiliza el lenguaje *Cypher* y soporta transacciones ACID.
- **Apache TinkerPop**: Framework para trabajar con estructuras de grafos mediante el lenguaje *Gremlin*. Sirve como base para diferentes sistemas de bases de datos de grafos.
- **JanusGraph**: Base de datos de grafos distribuida diseñada para manejar grandes volúmenes de datos. Puede utilizar sistemas como Cassandra o HBase para su almacenamiento.
- **Amazon Neptune**: Servicio administrado de bases de datos de grafos en la nube de AWS. Está diseñado para aplicaciones que necesitan trabajar con relaciones complejas a gran escala.
- **OrientDB**: Sistema de base de datos multimodelo que combina características de bases documentales y de grafos, permitiendo trabajar con ambos tipos de datos.
- **ArangoDB**: Base de datos multimodelo que permite trabajar con documentos, pares clave-valor y grafos. Utiliza *AQL* para realizar consultas sobre los diferentes modelos.